# 05L2-ML Basics

Generated by Scribe.AI

December 18, 2024

## Slide 1

### Content

**CS 24300 - AI Basics**
**Tony Bergstrom**
Slides adapted from Yexiang Xue

### Description

This slide is a title slide for a lecture on Artificial Intelligence Basics, likely part of a course called CS 24300. It displays the course title, the instructor's name (Tony Bergstrom), and a credit line indicating that the slides are adapted from another source (Yexiang Xue). The background is a dark gray, and the text is light gray, making it easily readable. The format is standard for a presentation slide, providing basic identifying information for the lecture.

### Figures

# Slide 2

## Content

**Experience Machine Learning**

## Description

This slide, likely the second slide of a presentation on Artificial Intelligence Basics, displays a single heading: "Experience Machine Learning." The text is centered on a light gray background. The context suggests that the lecture is about machine learning and how experience plays a role in the learning process. The title is likely an introduction to a section or topic within the broader subject of AI.

## Figures

# Slide 3

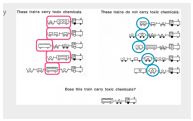
## Content

**What is machine learning?**
Machine learning (ML) is basically automated inductive reasoning.
**What is inductive reasoning?**

## Description

This slide, likely part of a presentation on Artificial Intelligence Basics, introduces the concept of machine learning. It defines machine learning (ML) as automated inductive reasoning. The slide also poses the question "What is inductive reasoning?" Visually, the slide features a series of diagrams depicting trains. Some trains are labeled "These trains carry toxic chemicals" and others "These trains do not carry toxic chemicals." A final train is shown, with the question "Does this train carry toxic chemicals?" The diagrams and question suggest that the slide is illustrating the process of inductive reasoning, where patterns are identified from examples to make predictions about unseen cases. The context of the course is crucial for understanding the connection between inductive reasoning and machine learning.

## Figures



**(a)**

# Slide 4

## Content

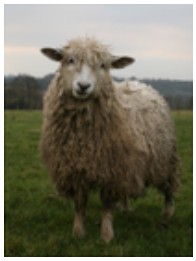
**Another example**
**Sheep**
**Horses**
**What about this?**
**Difficulty**: how to teach
**computers to do this?**

## Description

This slide presents an example, likely in a machine learning or computer vision context. It shows images of sheep and horses, and asks "What about this?". A final image of an alpaca is shown. The text "Difficulty: how to teach computers to do this?" suggests that the slide is illustrating a challenge in teaching computers to recognize or classify these animals, or more generally, to perform image recognition tasks. The context of the course (Artificial Intelligence Basics) indicates that the slide is introducing a concept related to image recognition, pattern recognition, or a similar topic in machine learning.

## Figures

**(a)**



**(b)**

**(c)**

**(d)**

**(e)**

# Slide 5

## Content

**Building a model from past examples**
Most machine learning builds a model from numerous data examples.
New examples are labeled/evaluated based on what has been seen before.
Data should be representative of the problems space one means to tackle.

## Description

This slide, likely part of a presentation on machine learning, discusses the fundamental process of building a machine learning model. It states that most machine learning methods construct a model from a collection of examples. Crucially, it emphasizes that new, unseen examples are evaluated based on the patterns learned from the training data. Finally, the slide highlights the importance of the training data being representative of the problem domain to be solved. The context of the course (Artificial Intelligence Basics) suggests that this slide is introducing the concept of supervised learning, where labeled data is used to train a model to make predictions on new, unlabeled data.

## Figures

# Slide 6

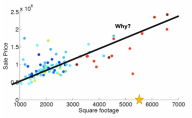
## Content

**The very first ML problem – linear regression**
House prices vs. square footage
[ xlabel=Square footage, ylabel=Sale Price, xmin=0, xmax=7000, ymin=0, ymax=2.5e+6, xtick=0,1000,2000,3000,4000,5000,
ytick=0,0.5e+6,1e+6,1.5e+6,2e+6,2.5e+6, legend pos=south east, ] [scatter, only marks, mark=*, mark size=1.5pt, color=blue!50!cyan] table x y 1200 0.8e+6 1500 1.0e+6 1800 1.2e+6 2000 1.3e+6 2500 1.5e+6 3000 1.6e+6 3500 1.7e+6 4000 1.8e+6 4500 1.9e+6 5000 2.0e+6 ; [scatter, only marks, mark=*, mark size=1.5pt, color=red!50!orange] table x y 1200 0.9e+6 1500 1.1e+6 1800 1.3e+6 2000 1.4e+6 2500 1.6e+6 3000 1.7e+6 3500 1.8e+6 4000 1.9e+6 4500 2.0e+6 5000 2.1e+6 ; [domain=0:7000, samples=100, color=black, thick] 0.0003*x + 0.5e+6; [mark=*, mark size=2pt, color=yellow!80!orange] coordinates (5500, 2.2e+6); [below right] at (axis cs:5000, 0.6e+6) But why this line?; [above right] at (axis cs:5000, 2e+6) Why?;

## Description

This slide introduces the concept of linear regression as the "very first ML problem." It displays a scatter plot illustrating the relationship between house prices (on the y-axis, in units of $10^6$) and square footage (on the x-axis). The plot shows numerous data points, each representing a house sale, with varying colors representing different datasets. A star-shaped marker is placed on a data point, likely to draw attention to a specific house sale or to illustrate a point about the model's fit. Th

## Figures




**(a)**

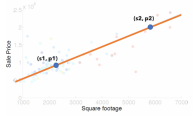
# Slide 7

## Content

## Description

This slide introduces the concept of linear regression as the "very first ML problem." It presents a scatter plot showing a set of data points, each represented by an ordered pair $(s, p)$, where $s$ represents square footage and $p$ represents sale price. Two specific points, $(s_1, p_1)$ and $(s_2, p_2)$, are highlighted on the plot. The slide asks how to find the line that passes through these two points. It then proposes a linear equation $p = ws + l$ to represent the line, where $w$ is the slope and $l$ is the y-intercept. The slide further explains that since the two points lie on the line, their coordinates satisfy the equation, leading to two linear equations: $p_1 = ws_1 + l$ and $p_2 = ws_2 + l$. Solving these two equations simultaneously will yield the values of $w$ and $l$, defining the line. The final question, "Seems we have way more than two points, what to do??", suggests that the slide is transitioning to the need for a method to handle more than two data points, which is a key aspect of machine learning where models are trained on large datasets. The context of the course (Artificial Intelligence Basics) indicates that this slide is introducing the fundamental concept of linear regression as a simple machine learning problem.

## Figures




**(a)**

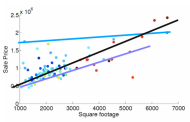
# Slide 8

## Content

## Description

This slide, part of a presentation on Artificial Intelligence Basics, introduces linear regression as the first machine learning problem. It displays a scatter plot showing the relationship between house prices (on the y-axis, in units of $10^6$) and square footage (on the x-axis). Multiple lines are plotted on the graph, including a black line that appears to be the best fit. The slide poses questions about finding the best-fitting line and how to define "best." It also asks why certain other lines (presumably the blue ones) are considered worse fits. The context suggests that the slide is introducing the concept of model selection and evaluation in machine learning, specifically focusing on linear regression.

## Figures




**(a)**

# Slide 9

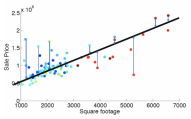
## Content

**The very first ML problem – linear regression**
Idea: find the line closest to all points!
How do you define "close"?
Given points $(s_1, p_1), \ldots, (s_N, p_N)$,
Sum of total distances from the line:
$(ws_1 + l - p_1)^2 + \cdots + (ws_N + l - p_N)^2$
Find the line to minimize this distance!
$\min_{w,l}(ws_1 + l - p_1)^2 + \cdots + (ws_N + l - p_N)^2$

## Description

This slide introduces linear regression as the first machine learning (ML) problem. It visually depicts a scatter plot (though not included in the LaTeX) showing data points representing various house sales, with square footage on the x-axis and sale price on the y-axis. The slide's text focuses on the core concept of finding the "closest" line to all the data points. It defines "closeness" as the sum of the squared distances between each data point and the line. The equation $\min_{w,l}(ws_1 + l - p_1)^2 + \cdots + (ws_N + l - p_N)^2$ is presented, highlighting the objective of finding the line parameters $w$ (slope) and $l$ (y-intercept) that minimize this sum of squared errors. The context of the course (Artificial Intelligence Basics) indicates that this slide is introducing a fundamental concept in regression analysis, where a line is fitted to data to model a relationship between variables.

## Figures




**(a)**

# Slide 10

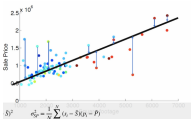
## Content

**The very first ML problem – linear regression**
We ask, which $w, l$ minimizes this distance?
$r = (ws_1 + l - p_1)^2 + \cdots + (ws_N + l - p_N)^2$
Minimizing a quadratic function... Calculus!
$\frac{\partial r}{\partial w} = 2s_1(s_1 w + l - p_1) + \cdots + 2s_N(s_N w + l - p_N) = 0$
$\frac{\partial r}{\partial l} = 2(s_1 w + l - p_1) + \cdots + 2(s_N w + l - p_N) = 0$
Gives
$w^* = \frac{\sigma_{SP}}{\sigma_{SS}^2}, \quad l^* = \bar{P} - w^* \bar{S}$
Where $\bar{P} = \frac{1}{N}\sum_{i=1}^{N} p_i$ and $\bar{S} = \frac{1}{N}\sum_{i=1}^{N} s_i$
$\sigma_{SP} = \frac{1}{N}\sum_{i=1}^{N}(s_i - \bar{S})(p_i - \bar{P})$
$\sigma_{SS}^2 = \frac{1}{N}\sum_{i=1}^{N}(s_i - \bar{S})^2$

## Description

This slide presents the derivation of the linear regression model's parameters. It's part of a lecture on Artificial Intelligence Basics, focusing on the fundamental concept of linear regression as a machine learning problem. The slide shows the objective function, $r$, which represents the sum of squared errors between the predicted values ($ws_i + l$) and the actual values ($p_i$) for $N$ data points. The equations $\frac{\partial r}{\partial w} = 0$ and $\frac{\partial r}{\partial l} = 0$ represent the necessary conditions for finding the minimum of this quadratic function using calculus. The resulting equations for $w^*$ and $l^*$ are shown, which are the optimal values for the slope and y-intercept of the best-fit line. The slide also defines the necessary variables, including $\bar{P}$ (average sale price), $\bar{S}$ (average square footage), $\sigma_{SP}$ (covariance between square footage and sale price), and $\sigma_{SS}^2$ (variance of square footage). The equations for these variables are explicitly given, showing how they are calculated from the data points. The context of the slide is crucial for understanding the derivation of the linear regression model's parameters, which are used to predict sale prices based on square footage.

## Figures



**(a)**

# Slide 11

## Content

## Description

This slide, part of a presentation on Artificial Intelligence Basics, introduces multiple linear regression. It extends the single-variable linear regression model to encompass scenarios with multiple independent variables. The formal mathematical form is presented, showing how a dependent variable ($y_i$) is related to multiple independent variables ($x_{i1}, x_{i2}, \ldots, x_{ip}$) through a linear equation. The equation $y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \epsilon_i$ is shown, where $\beta_0, \beta_1, \ldots, \beta_p$ are the coefficients to be estimated, and $\epsilon_i$ represents the error term. The slide then presents the equivalent matrix notation, $\mathbf{y} = \mathbf{X}\beta + \epsilon$, where $\mathbf{y}$ is the vector of dependent variables, $\mathbf{X}$ is the design matrix containing the independent variables, $\beta$ is the vector of coefficients, and $\epsilon$ is the vector of error terms. The slide defines the matrices $\mathbf{y}$, $\mathbf{X}$, $\beta$, and $\epsilon$ explicitly. Crucially, the slide concludes by stating the formula for calculating the optimal coefficients $\beta^*$ that minimize the sum of squared errors, which is $\beta^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$. A table of car data (Toyota, Mitsubishi, Skoda, Fiat) with their models, volume, weight, and CO2 emissions is included, illustrating the use of multiple independent variables in a real-world example. The context of the course is crucial for understanding the transition from simple linear regression to the more complex multiple linear regression model.

## Figures

**(a)** **(b)** **(c)** **(d)** **(e)**

# Slide 12

## Content

**Put it in Python**
import pandas
from sklearn import linear_model
df = pandas.read_csv("data.csv")
X = df[['Weight', 'Volume']]
y = df['CO2']
regr = linear_model.LinearRegression()
regr.fit(X, y)
predict the CO2 emission of a car where the weight is 2300kg, and the volume is 1300cm3:
predictedCO2 = regr.predict([[2300, 1300]])
print(predictedCO2)

## Description

This slide presents Python code for performing linear regression on a dataset. The code snippet imports the necessary libraries (‘pandas‘ and ‘linear$_m odel‘ from‘ sklearn‘).It loads a CSV file named "data.csv" into a pandas DataFrame (‘d

## Figures

# Slide 13

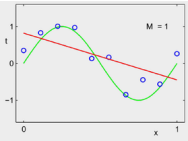
## Content

**What if we fit a curve?**
Given the blue points.
Ordinary linear regression gives the red line.
Fit this function (green curve):
$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$
Treat $(x, x^2, x^3)$ as the new independent variables!

## Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses extending linear regression to fit a curve instead of a straight line. It shows a scatter plot (blue points) and a straight line (red line) representing the result of ordinary linear regression. The slide proposes fitting a curve (green line) represented by the equation $y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$. The key idea is to treat the original independent variable $x$, along with its squared and cubed values ($x^2$ and $x^3$), as new independent variables in a multiple linear regression model. This allows for a more flexible fit to the data, potentially capturing non-linear relationships. The context of the lecture is on the limitations of linear regression and the need for more complex models to capture non-linear patterns in data.

## Figures




**(a)**

# Slide 14

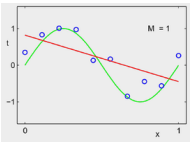
## Content

**What if I fit this with a degree-9 polynomial?**

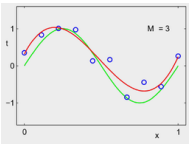
## Description

This slide, part of a presentation on Artificial Intelligence Basics, explores the concept of overfitting in polynomial regression. It displays three graphs. Each graph shows a set of scattered data points (blue circles) and three curves: a red curve representing a polynomial fit with a low degree (M=1), a green curve representing a polynomial fit with a medium degree (M=3), and a red curve representing a polynomial fit with a high degree (M=9). The graphs illustrate how increasing the degree of the polynomial (in this case, from M=1 to M=9) can lead to a curve that fits the training data extremely well (the green curve for M=9), but may not generalize well to unseen data. The graph for M=9 shows a highly oscillatory curve that closely follows the training data points, but deviates significantly from the general trend of the data. This demonstrates the potential for overfitting when using high-degree polynomials to model data. The context of the course is on the importance of model selection and avoiding overfitting in machine learning.

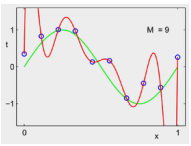
## Figures




**(a)**




**(b)**




**(c)**

# Slide 15

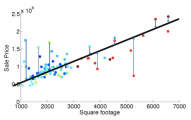
## Content

**Recap linear regression**
Step 1: pick a set of models
$F = \{f(x) = w^T x | w \in \mathbb{R}^D\}$
Step 2: define error / loss $L(y, y')$
$L(y, y') = (y - y')^2$
Step 3: minimize the loss
(empirical risk minimizer)
$f^* = \arg\min_{f \in F} \sum_{i=1}^{N} L(f(x_i), y_i)$

## Description

This slide summarizes the steps involved in linear regression. It's part of a lecture on Artificial Intelligence Basics. The slide outlines the three key steps: 1) choosing a set of models, represented by the equation $F = \{f(x) = w^T x | w \in \mathbb{R}^D\}$, where $f(x)$ is a linear function of $x$ and $w$ is a vector of weights; 2) defining a loss function, in this case, the squared error $L(y, y') = (y - y')^2$, measuring the difference between the predicted value $y'$ and the actual value $y$; and 3) finding the model that minimizes the total loss across all data points, represented by the equation $f^* = \arg\min_{f \in F} \sum_{i=1}^{N} L(f(x_i), y_i)$. A graph of a linear regression model fitted to data points is also included, showing the relationship between square footage and sale price. The context of the lecture is on the practical application of linear regression as a machine learning technique.

## Figures




**(a)**

# Slide 16

## Content

**What if multiple independent variables?**
Formal mathematical form:
$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \epsilon_i = x_i^T \beta + \epsilon_i$, for $i = 1, \ldots, n$
Or using matrix notation:
$y = X\beta + \epsilon$
Where:

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, X = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{pmatrix} = \begin{pmatrix} 1 & x_{11} & x_{12} & \ldots & x_{1p} \\ 1 & x_{21} & x_{22} & \ldots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \ldots & x_{np} \end{pmatrix}, \beta = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix}, \epsilon = \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

Can prove the optimal $\beta^*$ which minimizes the distances is: $\beta^* = (X^T X)^{-1} X^T y$

## Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses extending linear regression to handle multiple independent variables. It introduces the formal mathematical representation of multiple linear regression, where each dependent variable ($y_i$) is modeled as a linear combination of multiple independent variables ($x_{i1}, x_{i2}, \ldots, x_{ip}$) plus an error term ($\epsilon_i$). The slide then presents the equivalent matrix notation, showing how the multiple independent variables are represented in the matrix $X$. The equation $y = X\beta + \epsilon$ concisely expresses the relationship between the dependent variable vector $y$, the independent variable matrix $X$, the coefficient vector $\beta$, and the error vector $\epsilon$. A table of data (likely car models, volume, weight, and CO2 emissions) is included, illustrating the use of multiple independent variables in a real-world example. The slide concludes by stating that the optimal solution for the coefficients ($\beta^*$) can be found by minimizing the distances using the formula $\beta^* = (X^T X)^{-1} X^T y$. The context of the lecture is on the generalization of linear regression models to handle more complex relationships between variables.

## Figures

**(a)** **(b)** **(c)** **(d)** **(e)**

# Slide 17

## Content

**Work on a specific example**
Let's predict CO2 from Volume and Weight:
$y_i(CO2) = \beta_0 + \beta_1 V_i + \beta_2 W_i$
Can be written as:

$$y_i = \begin{pmatrix} 1 & V_i & W_i \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}$$

Hence, the four data points can be written as:

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} = \begin{pmatrix} 1 & V_1 & W_1 \\ 1 & V_2 & W_2 \\ 1 & V_3 & W_3 \\ 1 & V_4 & W_4 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}$$

## Description

This slide, part of a lecture on Artificial Intelligence Basics, demonstrates a specific example of multiple linear regression. It shows how to represent a set of data points (car models, volume, weight, and CO2 emissions) in a matrix form suitable for linear regression. The slide starts by defining the relationship between CO2 emissions ($y_i(CO2)$), volume ($V_i$), and weight ($W_i$) as a linear equation: $y_i(CO2) = \beta_0 + \beta_1 V_i + \beta_2 W_i$. This equation is then rewritten in matrix form, showing how each data point's features (1, $V_i$, $W_i$) are combined with the coefficients ($\beta_0$, $\beta_1$, $\beta_2$) to predict the CO2 emission ($y_i$). Finally, the slide presents the matrix representation of the four data points, where the dependent variable ($y_i$) is a column vector, and the independent variables (1, $V_i$, $W_i$) are arranged in a matrix ($X$) multiplied by the coefficient vector ($\beta$). The context of the lecture is on the practical application of multiple linear regression to model relationships between multiple variables.

## Figures

**(a)** **(b)** **(c)** **(d)** **(e)**

# Slide 18

## Content

**Work on a specific example**

Hence, the four data points can be written as:

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} = \begin{pmatrix} 1 & V_1 & W_1 \\ 1 & V_2 & W_2 \\ 1 & V_3 & W_3 \\ 1 & V_4 & W_4 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix} = \mathrm{X}\beta$$

$$\mathrm{X}^T X = \begin{pmatrix} 4 & \sum_{i=1}^{4} V_i & \sum_{i=1}^{4} W_i \\ \sum_{i=1}^{4} V_i & \sum_{i=1}^{4} V_i^2 & \sum_{i=1}^{4} V_i W_i \\ \sum_{i=1}^{4} W_i & \sum_{i=1}^{4} V_i W_i & \sum_{i=1}^{4} W_i^2 \end{pmatrix}$$

$$\beta^* = (X^T X)^{-1} X^T y$$

## Description

This slide, part of a lecture on Artificial Intelligence Basics, focuses on a specific example of multiple linear regression. It shows how to represent four data points in matrix form, suitable for calculating the optimal coefficients ($\beta^*$) using the formula $\beta^* = (X^T X)^{-1} X^T y$. The slide explicitly defines the matrix $X$ as a 4x3 matrix, where the first column represents a constant 1 for each data point, the second column represents the values of $V_i$, and the third column represents the values of $W_i$. The dependent variable $y_i$ is represented as a column vector. The slide also provides the matrix representation of $X^T X$, which is a 3x3 matrix containing sums of products of the independent variables. The formula for $\beta^*$ is presented, highlighting the crucial step of calculating the inverse of $X^T X$ and multiplying it by $X^T y$ to obtain the optimal coefficients. The context of the lecture is on the practical application of multiple linear regression to model relationships between multiple variables, using a specific example to illustrate the matrix calculations involved.

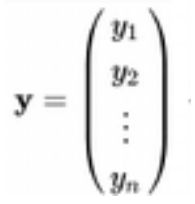
## Figures



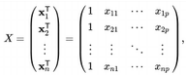
**(a)**



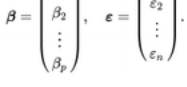
**(b)**



**(c)**



**(d)**



**(e)**

# Slide 19

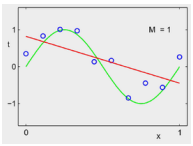
## Content

**What if I fit this with a degree-9 polynomial?**

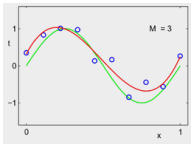
## Description

This slide, part of a lecture on Artificial Intelligence Basics, poses a question about fitting a dataset with a degree-9 polynomial. It likely accompanies graphs showing the results of fitting data with polynomials of different degrees (e.g., degree 1, degree 3, and degree 9). The graphs display the data points (represented by blue circles) and the fitted polynomial curves (in green and red). The slide's title suggests a comparison of the fit quality achieved by a degree-9 polynomial compared to lower-degree polynomials. The context of the lecture is likely on curve fitting, overfitting, or the impact of polynomial degree on model complexity and accuracy.

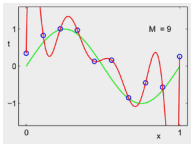
## Figures




**(a)**




**(b)**




**(c)**

# Slide 20

## Content

**Binary Classification**

## Description

This slide, likely part of a presentation on Artificial Intelligence Basics, simply displays the title "Binary Classification." It indicates that the following content will be related to the topic of binary classification tasks in machine learning.

## Figures

# Slide 21

## Content

## Description

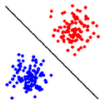
This slide introduces the concept of classification in the context of Artificial Intelligence Basics. It defines the setup for a binary classification problem. The input is a feature vector $x$ belonging to a D-dimensional real space ($\mathbb{R}^D$). The output is a label $y$ belonging to a set of classes $[C]$, which in this case, is a set of integers from 1 to C. The goal is to learn a mapping $f$ that takes the input feature vector $x$ and maps it to a class label $y$. The slide explicitly states that this particular lecture focuses on binary classification, meaning there are only two classes ($C = 2$). The labels are represented as $\{-1, +1\}$, which is a common representation for binary classification problems (e.g., -1 for "not fraud," +1 for "fraud"). Images of sheep and a horse are included, likely to illustrate the concept of classifying different types of animals. A scatter plot showing two clusters of points (red and blue) separated by a line is also present, visually representing a binary classification task. The context of the lecture is on machine learning algorithms for classifying data points into predefined categories.

## Figures

**(a)**

**(b)**

**(c)**



**(d)**

# Slide 22

## Content

**Recap linear regression**

Step 1: pick a set of models $F = \{f(x) = w^T x | w \in \mathbb{R}^D\}$

Step 2: define error / loss $L(y, y')$ $L(y, y') = (y - y')^2$

Step 3: minimize the loss (empirical risk minimizer) $f^* = \arg\min_{f \in F} \sum_{i=1}^{N} L(f(x_i), y_i)$

## Description

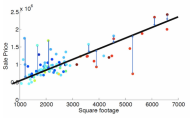
This slide summarizes the steps involved in linear regression. It's part of a lecture on Artificial Intelligence Basics, focusing on the practical application of linear regression. The slide lists three key steps:

1. **Step 1: pick a set of models:** This step defines the set of linear models to consider. The notation $F = \{f(x) = w^T x | w \in \mathbb{R}^D\}$ indicates that the models are linear functions of the input vector $x$, where $w$ is a vector of weights and $x$ is a vector of features. The set $F$ represents the space of all possible linear models.

2. **Step 2: define error / loss $L(y, y')$:** This step defines the error function used to measure the difference between the predicted value $y'$ and the actual value $y$. The loss function $L(y, y') = (y - y')^2$ is the squared error, a common choice for regression problems.

3. **Step 3: minimize the loss (empirical risk minimizer):** This step describes the process of finding the best model from the set $F$. The notation $f^* = \arg\min_{f \in F} \sum_{i=1}^{N} L(f(x_i), y_i)$ indicates that the optimal model $f^*$ is the one that minimizes the sum of squared errors across all training data points. $N$ represents the number of data points in the training set. The slide also includes a graph of a scatter plot with a fitted linear regression line, visually representing the process of finding the best-fitting line through the data points. The x-axis is labeled "Square footage" and the y-axis is labeled "Sale Price". The graph shows the relationship between square footage and sale price of houses, and the fitted line represents the linear regression model.

## Figures




**(a)**

# Slide 23

## Content

**Binary Classification**

What do you think, is a Linear Model still ok?

## Description

This slide, part of a presentation on Artificial Intelligence Basics, displays the title "Binary Classification." Below the title, a question is posed: "What do you think, is a Linear Model still ok?" This suggests that the subsequent content will discuss whether a linear model is still a suitable approach for binary classification tasks. The context implies a discussion about the limitations of linear models in more complex classification scenarios, potentially leading to a comparison with other models or a discussion of alternative approaches.

## Figures

# Slide 24

## Content

**Following the recipe: what is the set of models?**

**Step 1: select the set of models**

**Linear models seem Ok:**

$w^T x = w_1 x_1 + w_2 x_2$

**How can we modify the output?**

**Let's use the sign of $w^T x$ to predict the label**

$$\text{sign}(w^T x) = \begin{cases} +1 & \text{if } w^T x > 0 \\ -1 & \text{if } w^T x \leq 0 \end{cases}$$

**Why is this visually a line?**

## Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses binary classification using linear models. It presents a step-by-step approach to building a classifier.

The slide begins by asking "Following the recipe: what is the set of models?". This implies a previous discussion about a "recipe" or algorithm for building a classifier.

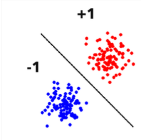
The first step is to select the set of models. The slide states that linear models are suitable, and defines the model as $w^T x = w_1 x_1 + w_2 x_2$. This is a linear equation in two dimensions (assuming $x_1$ and $x_2$ are the input features).

The slide then asks how to modify the output of the linear model to predict the class labels. It proposes using the sign of $w^T x$ to predict the label. The sign function is defined as:

$$\text{sign}(w^T x) = \begin{cases} +1 & \text{if } w^T x > 0 \\ -1 & \text{if } w^T x \leq 0 \end{cases}$$

Finally, the slide asks "Why is this visually a line?". This suggests that the linear model, when used with the sign function, will produce a decision boundary that is a line in the two-dimensional feature space. The slide likely contains a visual representation of this decision boundary, separating the data points into two classes. The context of the lecture is on machine learning algorithms for classifying data points into predefined categories, specifically binary classification.

## Figures




**(a)**

# Slide 25

## Content

**Following the recipe: loss function**
**Step 2: define loss function $L(y, y')$**
**What does it mean that we have an error?**

- True label $+1$, predicted $-1$: $y = 1$, $y' = \text{sign}(w^T x) = -1$
- True label $-1$, predicted $+1$: $y = -1$, $y' = \text{sign}(w^T x) = +1$

**Concisely represented as 0-1 loss:**
$L(y, y') = 1[y \neq y']$
**Or more explicitly as:**
$l_{01}(z) = 1[z \leq 0]$ where $z = y(w^T x)$

## Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses the definition of a loss function for binary classification using linear models. The slide is titled "Following the recipe: loss function" and focuses on Step 2 of a recipe-like approach to building a classifier.

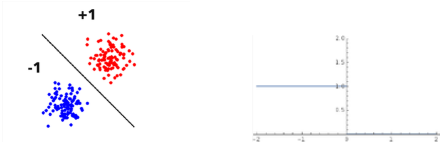
The slide introduces the concept of a loss function $L(y, y')$ to quantify the error in a prediction. It defines the error in terms of the true label ($y$) and the predicted label ($y'$). The true label is either $+1$ or -1, and the predicted label is determined by the sign of the linear model's output, $w^T x$. The slide shows two cases where an error occurs:

1. The true label is $+1$, but the predicted label is -1. 2. The true label is -1, but the predicted label is $+1$.

The slide then concisely represents this error as a 0-1 loss function: $L(y, y') = 1[y \neq y']$. This function returns 1 if the predicted label ($y'$) does not match the true label ($y$), and 0 otherwise. The slide also provides a more explicit representation of the loss function using an indicator function: $l_{01}(z) = 1[z \leq 0]$, where $z = y(w^T x)$. This shows how the loss depends on the output of the linear model.

The slide includes a scatter plot showing data points classified into two categories ($+1$ and -1) and a decision boundary. The context of the lecture is on machine learning algorithms for classifying data points into predefined categories, specifically binary classification. The slide is likely part of a larger discussion on how to train a linear classifier.

## Figures




**(a)** **(b)**

# Slide 26

## Content

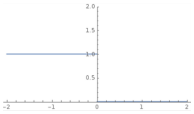
**0-1 loss?**

Good idea? What could go wrong?
No gradient! Almost impossible to minimize!
How can we fix it?

## Description

This slide, part of a presentation on Artificial Intelligence Basics, discusses the 0-1 loss function for binary classification. The title "0-1 loss?" poses a question about the suitability of this loss function. The subsequent text critiques the 0-1 loss function by stating that it lacks a gradient, making it difficult to minimize using gradient-based optimization methods. A graph of a horizontal line at $y = 1$ is included, visually representing the constant value of the loss function. The context suggests a discussion about alternative loss functions that are more amenable to optimization algorithms, such as gradient descent, in the context of training a binary classifier.

## Figures



**(a)**

# Slide 27

## Content

**What we can do...**

**Perceptron loss:** $l(z) = \max\{0, -z\}$
**Hinge loss:** $l(z) = \max\{0, 1 - z\}$
**Logistic loss:** $l(z) = \log(1 + e^{-z})$
**Reminder:** $z = y(w^T x)$

## Description

This slide presents three different loss functions for a binary classification model, likely in the context of training a linear classifier. The slide is titled "What we can do...", suggesting a discussion of alternative approaches to the previous loss function.

The three loss functions are:

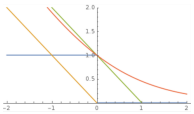
1. **Perceptron loss**: $l(z) = \max\{0, -z\}$. This is a piecewise linear function that penalizes misclassifications by a margin.
2. **Hinge loss**: $l(z) = \max\{0, 1 - z\}$. This is another piecewise linear loss function, also penalizing misclassifications by a margin. The difference is the threshold for the margin is 1.
3. **Logistic loss**: $l(z) = \log(1 + e^{-z})$. This is a smooth, non-linear loss function that is commonly used in logistic regression.

The slide also includes a graph depicting the shapes of these loss functions. The graph shows how the loss changes as the input $z$ varies. The graph is crucial for understanding the behavior of each loss function.

The reminder $z = y(w^T x)$ indicates that $z$ is the output of the linear model, where $y$ is the true label and $w^T x$ is the prediction. This is a key component for understanding how the loss functions are applied in the context of the model.

The context of the course (Artificial Intelligence Basics) suggests that the slide is part of a discussion on different ways to train a binary classifier, comparing the properties and characteristics of various loss functions.

## Figures



**(a)**

# Slide 28

## Content

**Empirical risk minimizer**

**Step 3: minimize the loss:**

$$w^* = \arg\min_{w \in \mathbb{R}^d} \sum_{i=1}^{N} l(y_i, w^T x_i)$$

**How?**

Follow the gradient direction!

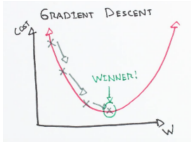
## Description

This slide, part of a lecture on Artificial Intelligence Basics, discusses the third step in a process for empirical risk minimization. The title is "Empirical risk minimizer." The step is to minimize the loss function. The slide presents the mathematical expression for this minimization:

$$w^* = \arg\min_{w \in \mathbb{R}^d} \sum_{i=1}^{N} l(y_i, w^T x_i)$$

This equation finds the optimal weight vector $w^*$ that minimizes the sum of the loss $l(y_i, w^T x_i)$ for each data point $(x_i, y_i)$ in a dataset of size $N$. The variable $w$ belongs to the $d$-dimensional real space ($\mathbb{R}^d$). The slide then asks "How?" and answers with "Follow the gradient direction!". This implies that the method for finding the minimum involves following the direction of the gradient of the loss function. A graph is included, illustrating a cost function and the process of gradient descent, visually representing the concept of following the gradient to find the minimum. The context of the lecture is on machine learning algorithms, specifically how to train a model by minimizing a loss function.

## Figures




**(a)**

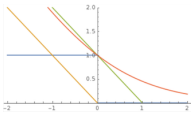
# Slide 29

## Content

## Description

This slide discusses the gradient of the perceptron loss function. It defines the perceptron loss as $l_{perceptron}(z) = \max\{0, -z\}$. The objective function $F(w)$ is the sum of the perceptron losses over all data points, where $w$ is the weight vector, $x_i$ is the input vector, and $y_i$ is the corresponding target value for the $i$-th data point. The slide then asks for the gradient of $F(w)$, denoted as $\nabla F(w)$. The answer is given as $\nabla F(w) = \sum_{i=1}^{N} -1[y_i w^T x_i \leq 0] y_i x_i$. This formula indicates that the gradient is a sum of terms, each corresponding to a data point. The term $[y_i w^T x_i \leq 0]$ is an indicator function, which is 1 if the inner product $y_i w^T x_i$ is less than or equal to 0, and 0 otherwise. The slide also describes a stochastic gradient descent approach, called "Feeling Lazy," for minimizing the loss function. The method involves randomly selecting a data point that is misclassified and updating the weights in the direction of the gradient for that single data point. A graph of the perceptron loss function is included, visually illustrating the concept of the loss function and its gradient. The context of the course (Artificial Intelligence Basics) suggests a discussion of training algorithms for binary classification models using the perceptron loss function.

## Figures



**(a)**

# Slide 30

## Content

**The Perceptron Algorithm**

**Repeat:**

Pick a data point $x_n$ uniformly at random

**If** $\text{sign}(w^T x_n) \neq y_n$ **then**

$w \leftarrow w + y_n x_n$

**If** the data is linearly separable, this algorithm will find the separating hyperplane!

## Description

This slide describes the Perceptron Algorithm, a fundamental algorithm in machine learning, specifically for binary classification. The algorithm is presented as a series of steps, which are repeated until a separating hyperplane is found.

The algorithm's core logic is:

1. **Repeat**: The algorithm iterates through the steps repeatedly.

2. **Pick a data point** $x_n$ **uniformly at random**: A data point ($x_n$) is randomly selected from the training dataset.

3. **If sign**$(w^T x_n) \neq y_n$ **then**: This is the crucial condition. It checks if the prediction of the current weight vector $w$ on the selected data point ($x_n$) is incorrect. The prediction is based on the sign of the dot product $w^T x_n$. The target value for the data point is $y_n$. If the prediction is wrong, the weight vector $w$ is updated.

4. $w \leftarrow w + y_n x_n$: If the prediction is incorrect, the weight vector $w$ is updated by adding the product of the target value ($y_n$) and the input vector ($x_n$).

5. **If the data is linearly separable, this algorithm will find the separating hyperplane!**: This statement highlights the algorithm's property. If the data points are linearly separable (meaning a straight line or hyperplane can perfectly separate the data points of different classes), the algorithm will eventually converge to a weight vector that defines that separating hyperplane.

The context of the course (Artificial Intelligence Basics) suggests that this slide is part of a discussion on supervised learning algorithms for binary classification problems, specifically introducing the Perceptron Algorithm.

## Figures

# Slide 31

## Content

**How about logistic loss?**

Find the minimizer of $F(w) = \sum_{i=1}^{N} l_{logistic}(y_i w^T x_i) = \sum_{i=1}^{N} \log(1 + e^{-y_i w^T x_i})$

It is actually called logistic regression

- Why logistic loss?
- Why regression?

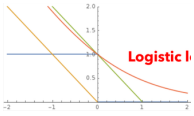
## Description

This slide, part of a lecture on Artificial Intelligence Basics, introduces the concept of logistic loss. The title is "How about logistic loss?". The slide presents the mathematical expression for the logistic loss function:

$$F(w) = \sum_{i=1}^{N} l_{logistic}(y_i w^T x_i) = \sum_{i=1}^{N} \log(1 + e^{-y_i w^T x_i})$$

This equation defines a function $F(w)$ that depends on the weight vector $w$ and aims to find the weight vector that minimizes this function. The function is a sum of logistic losses for each data point $(x_i, y_i)$ in a dataset of size $N$. The slide also states that this is equivalent to logistic regression. The slide then poses two questions: "Why logistic loss?" and "Why regression?". These questions are likely to be addressed in the subsequent parts of the lecture, exploring the rationale behind using this specific loss function and the connection to regression techniques in the context of machine learning. A graph of a logistic loss function is included, visually illustrating the shape of the loss function. The context of the course is on machine learning algorithms, specifically discussing different loss functions and their applications.

## Figures



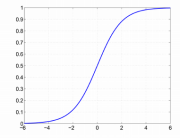

**(a)**

# Slide 32

## Content

## Description

## Figures



**(a)**

# Slide 33

## Content

**Perceptron vs. logistic regression**

**Perceptron Repeat:**

- Pick a data point $x_n$, uniformly at random
- If $\text{sign}(w^T x_n) \neq y_n$ then
- $w \leftarrow w + y_n x_n$

**Logistic Regression Repeat:**

- Pick a data point $x_n$, uniformly at random
- Update $w$ according to
- $w \leftarrow w + \sigma(-y_n w^T x_n) y_n x_n$

## Description

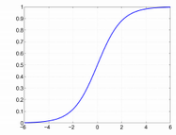
This slide compares the Perceptron algorithm and Logistic Regression, both used for binary classification. It presents the core update rules for each algorithm in a tabular format.

For the **Perceptron**, the algorithm repeats the following steps: 1. Randomly selects a data point $x_n$. 2. Checks if the prediction based on the current weight vector $w$ ($sign(w^T x_n)$) is incorrect compared to the target value $y_n$. 3. If the prediction is wrong, the weight vector $w$ is updated by adding the product of the target value ($y_n$) and the input vector ($x_n$).

For **Logistic Regression**, the algorithm also repeats the following steps: 1. Randomly selects a data point $x_n$. 2. Updates the weight vector $w$ using a more sophisticated update rule involving the sigmoid function ($\sigma$). The update rule is $w \leftarrow w + \sigma(-y_n w^T x_n) y_n x_n$. The sigmoid function ($\sigma$) is crucial here, and its presence in the update rule distinguishes Logistic Regression from the Perceptron. The sigmoid function maps the dot product $w^T x_n$ to a probability between 0 and 1, which is a key difference in how these algorithms handle classification.

The slide also includes a graph of the sigmoid function, visually illustrating its S-shaped curve. The context of the course (Artificial Intelligence Basics) suggests that this slide is part of a discussion on supervised learning algorithms for binary classification problems, comparing the Perceptron algorithm with the more sophisticated Logistic Regression algorithm.

## Figures



**(a)**